

SOFTWARE ARCHITECTURE

ASSIGNMENT 2, PART 2

Omnichannel Commerce Core

Architecture Report

Scenario C: nopCommerce Architectural Evolution

Guilherme Rosa

Rui Machado
Maria Linhares
Gabriel Silva

May 2026

Deadline: 02-03 June 2026

Contents

1	Introduction	6
1.1	Context and Scenario	6
1.2	The Architectural Bet	6
1.3	Report Structure	6
2	Architecture Evolution	8
2.1	What Remained Unchanged	8
2.2	Structural Change: Two Plugins Instead of One	8
2.3	New: Compensated Order State	9
2.4	New: Inbox Deduplication	11
2.5	Event Contracts Settled	11
2.6	Scope Cuts Carried Over From Part 1	11
3	Implementation Overview	12
3.1	OmnichannelOutbox Plugin	12
3.1.1	OutboxMessage Entity	12
3.1.2	OutboxEventPublisherDecorator	12
3.1.3	OrderProcessingServiceDecorator	13
3.1.4	OutboxDispatcherTask	13
3.1.5	OutboxRetentionTask	13
3.2	OmnichannelConsumers Plugin	13
3.2.1	RabbitMqSubscriberTask	14
3.2.2	InboxDeduplicationService	14
3.2.3	Event Handlers	14
3.2.4	CompensationService	14
3.3	Feature Flag: Synchronous Fallback	14
4	Architecture Decision Records	16

ADR-1: Transactional Outbox	16
ADR-2: Optimistic Stock Reservation	17
ADR-3: WireMock Carrier Simulation	17
ADR-4: Selective Context Extraction	17
ADR-5: Two Plugins Instead of One	18
ADR-6: Inbox Table Deduplication	18
5 Cross-Cutting Concerns	20
5.1 Idempotency	20
5.2 Error Handling and Dead-Letter Strategy	21
5.3 Observability	21
5.4 Security	22
5.5 Scalability Constraints	22
6 Traceability Matrix	23
6.1 Business Drivers	23
6.2 Quality Attribute Scenarios (Part 1 summary)	23
6.3 Full Traceability Matrix	24
6.4 Risk Coverage	25
7 Demo Orchestration	26
7.1 Prerequisites	26
7.2 Scenario 1 — Happy Path	26
7.3 Scenario 2 — Warehouse Degradation	27
7.4 Scenario 3 — Recovery	27
7.5 Scenario 4 — Forced Rejection and Compensation	28
7.6 Scenario 5 — Audit Trail	28
8 Tradeoffs, Scope Cuts, and Limitations	30
8.1 Tradeoffs We Own	30
8.1.1 At-Least-Once vs. Exactly-Once Delivery	30
8.1.2 Dispatcher Polling vs. DB Change Notifications	30
8.1.3 Whitelisted vs. Universal Outbox	30
8.1.4 In-Process Outbox vs. Separate Worker	31
8.2 Scope Cuts and Their Consequences	31
8.3 Known Limitations	31

8.4	What We Would Do Differently at Production Scale	32
9	Use of Large Language Models	33
9.1	Overview	33
9.2	Where LLMs Were Used	33
9.2.1	Documentation and Report Writing	33
9.2.2	Architecture Diagram Generation	33
9.2.3	Code Generation	34
9.2.4	Architectural Reasoning	34
9.3	Where LLMs Were Not Used	34
9.4	Reflection	34

List of Figures

2.1	Order lifecycle state machine including the new Compensated state	10
-----	--	----

List of Tables

2.1	Plugin split rationale	9
2.2	Cross-context event contracts	11
3.1	OutboxMessage schema	12
3.2	Inbound event handlers	14
5.1	Error handling strategy by failure point	21
6.1	Quality attribute scenarios	23
6.2	Business drivers, QA scenarios, and implementation artefacts	24
6.3	Risk register: implementation status	25
8.1	Scope cuts and their consequences	31

Chapter 1

Introduction

1.1 Context and Scenario

This report documents the Part 2 implementation of Scenario C, *Omnichannel Commerce Core*, for the Software Architecture assignment. The system under evolution is **nop-Commerce**, an open-source e-commerce platform, extended to coordinate reliably with external operational systems: an OpenBoxes warehouse management system (WMS), a WireMock-simulated carrier, and a RabbitMQ message broker.

The domain is modelled for **VerdeMart Retail**, a fictional retailer operating both online and physical channels.

The team is composed of Maria Linhares, Rui Machado, Guilherme Rosa, and Gabriel Silva.

1.2 The Architectural Bet

The central decision of Part 1, carried into implementation in Part 2, is stated in one sentence:

Decision

Replace the synchronous stock decrement inside `OrderProcessingService` with an outbox-published event, so order acceptance survives warehouse degradation and reconciles on recovery.

This single change, and everything it pulls along, is the assignment. It directly addresses **QA-1** (availability under warehouse degradation) and sets the foundation for **QA-2** (stock consistency) and **QA-3** (order placement throughput).

1.3 Report Structure

Chapter 2 Architecture evolution from Part 1 plan to actual implementation: what changed and why.

Chapter 3 Implementation overview of both plugins built inside the nopCommerce monolith.

Chapter 4 Architecture Decision Records: updated from Part 1 and new decisions made during implementation.

Chapter 5 Cross-cutting concerns: idempotency, error handling, observability, security.

Chapter 6 Traceability matrix linking business drivers to QA scenarios to implementation.

Chapter 7 Demo orchestration: the four scenarios and their success criteria.

Chapter 8 Defence of tradeoffs, scope cuts, and known limitations.

Chapter 2

Architecture Evolution

This chapter describes how the target architecture defined in Part 1 evolved during implementation. It covers what remained unchanged, what was refined, and what was cut.

2.1 What Remained Unchanged

The five bounded contexts identified in Part 1 held through implementation without revision:

1. **Catalog:** inside `nopCommerce`; owns product definitions, pricing, categories.
2. **Order Management:** inside `nopCommerce`; owns the order lifecycle, customer intent, payment authorisation.
3. **Identity:** inside `nopCommerce` (cross-cutting); owns customer accounts and authentication.
4. **Inventory / Warehouse:** extracted; owns authoritative stock truth, reservations, stock movements.
5. **Shipping / Fulfillment:** extracted; owns carrier dispatch, tracking, delivery state.

The **transactional outbox** as the reliability mechanism was also confirmed and not revisited. The feasibility spike (Part 1, §7) had already de-risked the critical constraint: `PlaceOrderAsync` does not run inside an ambient `TransactionScope`, so the outbox decorator must supply one.

2.2 Structural Change: Two Plugins Instead of One

Part 1 described a single plugin (`Nop.Plugin.Misc.OmnichannelOutbox`) carrying both the outbox write and the inbound RabbitMQ subscriber. During implementation, the inbound side was split into a **separate plugin**: `Nop.Plugin.Misc.OmnichannelConsumers`.

Table 2.1: Plugin split rationale

Plugin	Responsibility
<code>OmnichannelOutbox</code>	Outbox write path: entity, publisher decorator, dispatcher, RabbitMQ publisher, TransactionScope decorator, retention cleanup
<code>OmnichannelConsumers</code>	Inbound path: RabbitMQ subscriber, inbox deduplication, event handlers, compensation orchestration

This separation follows the single-responsibility principle at plugin granularity and mirrors the Part 1 ADD iteration structure (Iteration 1 = outbox, Iteration 2 = consumers and stock projection).

2.3 New: Compensated Order State

The `Compensated` state was added to the `OrderStatus` enum in `Nop.Core`. This was anticipated in Part 1 but is now concrete code. The state is reached when the Inventory service rejects a stock reservation and the compensation flow (payment release, customer notification) completes.

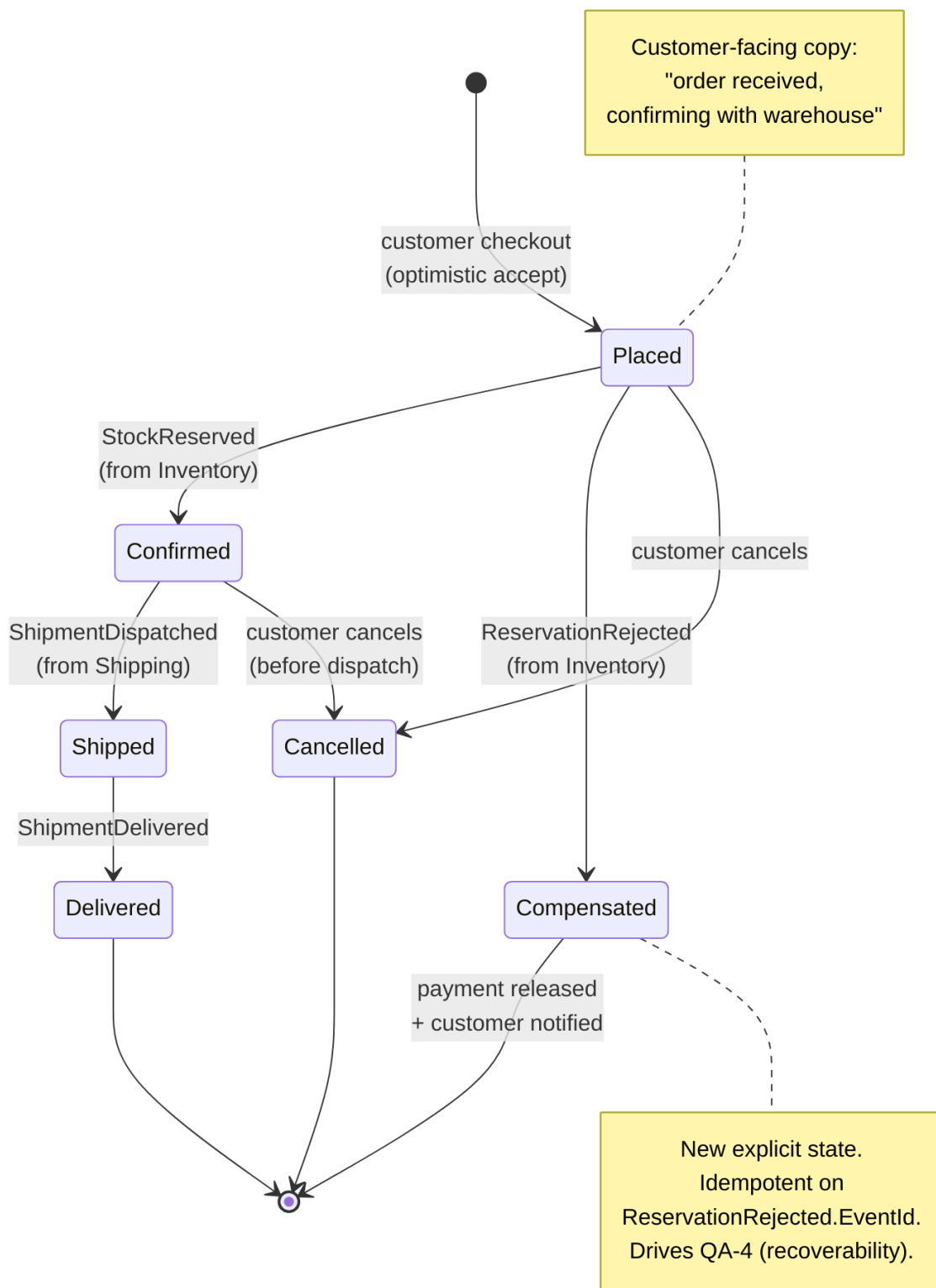


Figure 2.1: Order lifecycle state machine including the new **Compensated** state

2.4 New: Inbox Deduplication

Part 1 stated that consumers must be idempotent but left the mechanism open. The implementation chose an **inbox table** approach: a `ProcessedInboxMessage` entity records every `EventId` that has been handled. Before processing any inbound message, `InboxDeduplicationService` checks for an existing record. Duplicates are silently discarded.

This satisfies the at-least-once delivery guarantee without requiring exactly-once semantics from RabbitMQ.

2.5 Event Contracts Settled

The event contracts were implemented as dedicated event classes in `Nop.Core.Domain`:

Table 2.2: Cross-context event contracts

Event	Direction	Published when
<code>OrderPlacedEvent</code>	Outbound	Customer places an order
<code>StockReservedEvent</code>	Inbound	Inventory confirms stock reservation
<code>ReservationRejected-Event</code>	Inbound	Inventory cannot fulfil the reservation
<code>OrderConfirmedEvent</code>	Outbound	Order moves to Confirmed state
<code>StockLevelChangedEvent</code>	Inbound	External stock movement
<code>ShipmentDispatchedEvent</code>	Inbound	Shipping service dispatches the parcel
<code>OrderCompensatedEvent</code>	Internal	Compensation flow completes

2.6 Scope Cuts Carried Over From Part 1

All acknowledged scope cuts from Part 1 remain in force:

- Identity stays as-is (no federated SSO across channels).
- Payment is folded into Order Management, with no separate Payment service.
- Shipping equals Fulfillment: one extracted service, not two.
- Single `nopCommerce` instance: no Redis, no horizontal scaling.
- The outbox lives **inside the monolith process** as an `IScheduleTask`, not as a separate worker.

Chapter 3

Implementation Overview

3.1 OmnichannelOutbox Plugin

`Nop.Plugin.Misc.OmnichannelOutbox` is the outbox write path. It is registered via `INopStartup` and requires no changes to `nopCommerce` core.

3.1.1 OutboxMessage Entity

`OutboxMessage` is persisted to a dedicated table created by a `FluentMigrator` schema migration shipped with the plugin. Key fields:

Table 3.1: `OutboxMessage` schema

Column	Type	Purpose
<code>Id</code>	<code>int</code>	Primary key
<code>EventId</code>	<code>Guid</code>	Idempotency key for consumers
<code>MessageType</code>	<code>string</code>	Fully qualified event type name
<code>Payload</code>	<code>string</code>	JSON-serialised event payload
<code>OccurredOnUtc</code>	<code>DateTime</code>	When the event was written
<code>DispatchedOnUtc</code>	<code>DateTime?</code>	Null until the dispatcher confirms broker ack
<code>Attempts</code>	<code>int</code>	Incremented on each failed dispatch attempt

3.1.2 OutboxEventPublisherDecorator

Wraps `nopCommerce`'s `IEventPublisher`. For a **whitelisted set** of cross-context event types, it writes an `OutboxMessage` row instead of (or in addition to) calling in-process consumers. All other event types pass through unchanged, preserving compatibility with existing plugins (`Brevo`, `Omnisend`, etc.).

The whitelist is the explicit seam between bounded contexts. Adding a new cross-context event requires editing the whitelist — deliberate friction to prevent accidental coupling.

3.1.3 OrderProcessingServiceDecorator

Wraps `IOrderProcessingService` to supply the missing `TransactionScope` around the order-creation path. The decorator uses:

Listing 3.1: `TransactionScope` pattern used in the decorator

```
1 using var ts = new TransactionScope(  
2     TransactionScopeOption.Required,  
3     new TransactionOptions { IsolationLevel =  
4         IsolationLevel.ReadCommitted },  
5     TransactionScopeAsyncFlowOption.Enabled);  
6  
7 var result = await  
8     _inner.PlaceOrderAsync(processPaymentRequest);  
9  
10 if (result.Success)  
11     ts.Complete();
```

Two non-obvious constraints confirmed by the feasibility spike:

- `TransactionScopeAsyncFlowOption.Enabled` is mandatory: without it, `Transaction.Current` does not flow across the first `await` and the outbox row falls outside the transaction.
- Isolation level is tuned to `ReadCommitted` to avoid a hidden throughput tax from the default `Serializable` (QA-3 concern).

3.1.4 OutboxDispatcherTask

An `IScheduleTask` polling the outbox table on a 5-10 s cadence. For each undispatched row it publishes to RabbitMQ via `RabbitMqPublisher` and, on broker ack, sets `DispatchedOnUtc`. On failure it increments `Attempts`; rows exceeding a configured maximum remain in the table as a dead-letter view for manual inspection.

3.1.5 OutboxRetentionTask

A second `IScheduleTask` that deletes dispatched outbox rows older than 7 days, preventing unbounded table growth (Risk R5 mitigation).

3.2 OmnichannelConsumers Plugin

`Nop.Plugin.Misc.OmnichannelConsumers` is the inbound path. It subscribes to RabbitMQ, deduplicates messages, and routes them to the appropriate handler.

3.2.1 RabbitMqSubscriberTask

An `IScheduleTask` that drives the RabbitMQ consumer loop. On each tick it drains available messages from the configured queues and dispatches them to `IInboxMessageHandler` implementations.

3.2.2 InboxDeduplicationService

Before any handler runs, `InboxDeduplicationService` checks whether the message's `EventId` already exists in the `ProcessedInboxMessage` table. If it does, the message is silently discarded. This enforces idempotency at the infrastructure layer, freeing individual handlers from duplicating the check.

`ProcessedInboxMessage` is persisted via a dedicated schema migration in this plugin.

3.2.3 Event Handlers

Table 3.2: Inbound event handlers

Handler	Behaviour
<code>StockReservedHandler</code>	Moves the order from <code>Placed</code> to <code>Confirmed</code> ; publishes <code>Order-ConfirmedEvent</code> to the outbox for the Shipping service
<code>ReservationRejectedHandler</code>	Delegates to <code>CompensationService</code> : voids the payment, notifies the customer, transitions order to <code>Compensated</code>
<code>StockLevelChangedHandler</code>	Updates the Catalog stock projection (read model) inside the monolith
<code>ShipmentDispatchedHandler</code>	Advances the order to <code>Shipped</code> state and records the tracking reference

3.2.4 CompensationService

Orchestrates the compensation flow on reservation rejection:

1. Void or refund the payment authorisation.
2. Send a customer notification (via `nopCommerce`'s existing messaging infrastructure).
3. Transition the order status to `Compensated`.
4. Publish `OrderCompensatedEvent` (in-process, for audit trail).

3.3 Feature Flag: Synchronous Fallback

A feature flag controls whether the legacy synchronous stock decrement inside `OrderProcessingService` remains active. During the migration period this flag allows

the team to run both paths side-by-side and validate the asynchronous path before fully cutting over. This is the migration strategy for risk R2.

Chapter 4

Architecture Decision Records

This chapter presents the Architecture Decision Records (ADRs) for Part 2. ADRs 1 to 4 are carried over from Part 1 with implementation notes. ADRs 5 and 6 are new decisions made during implementation.

ADR-1: Transactional Outbox (carried over, status: Implemented)

Status: Accepted. Implemented in `OmnichannelOutbox` plugin.

Decision

Write cross-context events to an `OutboxMessage` table inside the same DB transaction as the order, then dispatch to RabbitMQ asynchronously via a polling `IScheduleTask`.

Implementation notes

The `TransactionScopeAsyncFlowOption.Enabled` constraint discovered in the feasibility spike was confirmed in production code. Isolation level was tuned to `ReadCommitted` as anticipated. No further divergence from the Part 1 decision.

Alternatives considered (Part 1)

- Direct RabbitMQ publish from `IConsumer<OrderPlacedEvent>`: REJECTED (dual-write risk).
- XA / two-phase commit: REJECTED (RabbitMQ not an XA resource; operational complexity).
- CDC / Debezium log mining: REJECTED (engine-specific; out of scope).
- Synchronous publish inside the order transaction: REJECTED (broker in critical path).

ADR-2: Optimistic Stock Reservation (carried over, status: Implemented)

Status: Accepted. Implemented via `OrderPlacedEvent` outbox path and `StockReservedHandler` / `ReservationRejectedHandler`.

Decision

Accept orders optimistically without a synchronous stock check. The `OrderPlaced` event instructs the Inventory service to attempt a reservation. If the reservation fails, the compensation flow transitions the order to `Compensated`.

Implementation notes

The `Compensated` state is now concrete in `OrderStatus`. The compensation flow is implemented in `CompensationService`. No divergence from the Part 1 decision.

ADR-3: WireMock Carrier Simulation (carried over, status: Implemented)

Status: Accepted. Implemented in the Shipping Service.

Decision

Use WireMock to simulate the carrier API, avoiding a dependency on a live external service during development and the demo.

ADR-4: Selective Context Extraction (carried over, status: Implemented)

Status: Accepted. Inventory and Shipping services extracted; Catalog, Order Management, and Identity remain in the monolith.

Decision

Extract only the contexts whose failure modes are operationally distinct (Inventory, Shipping). Keep the remaining contexts inside the `nopCommerce` monolith to avoid unnecessary operational complexity for a demo-scale system.

ADR-5: Two Plugins Instead of One (new)

Status: Accepted. Decided during Part 2 implementation.

Context

Part 1 described a single `OmnichannelOutbox` plugin. During implementation the outbox write path and the inbound subscriber path grew in different directions, with distinct failure domains and lifecycle concerns.

Decision

Split into two plugins: `OmnichannelOutbox` (write path) and `OmnichannelConsumers` (read/inbound path).

Rationale

- Each plugin has a single deployment and failure mode.
- The two paths can be enabled/disabled independently without affecting the other.
- Mirrors the ADD iteration structure: Iteration 1 = outbox write, Iteration 2 = consumers.

Alternatives considered

- **Single plugin:** REJECTED: one plugin holding both the dispatcher and the subscriber creates a large, hard-to-test unit with mixed failure domains.
- **Separate process for consumers:** REJECTED: out of scope; the monolith process is already hosting the outbox dispatcher as an `IScheduleTask`.

ADR-6: Inbox Table Deduplication (new)

Status: Accepted. Implemented in `InboxDeduplicationService` and `ProcessedInboxMessage` entity.

Context

RabbitMQ delivers at-least-once. Part 1 stated that consumers must be idempotent but left the mechanism open.

Decision

Persist a `ProcessedInboxMessage` record (keyed on `EventId`) for every handled message. Check for this record before processing. Duplicates are discarded without executing handler logic.

Rationale

- Handler logic (state transitions, payment voids, notifications) is not naturally idempotent; centralising the check at the infrastructure layer removes the burden from individual handlers.
- The `ProcessedInboxMessage` table doubles as an audit log of processed messages.
- The same `InboxRetentionTask` pattern used by the outbox cleans up old records.

Alternatives considered

- **Handler-level idempotency:** REJECTED: each handler would need to query its own state to determine whether the action had already been taken (e.g., “is this order already Confirmed?”). This is error-prone and forces domain logic to carry infrastructure concerns.
- **Exactly-once delivery via RabbitMQ quorum queues:** REJECTED: quorum queues reduce but do not eliminate duplicates; does not remove the need for consumer-side idempotency.

Chapter 5

Cross-Cutting Concerns

5.1 Idempotency

Idempotency is enforced at two levels:

1. **Outbox side:** each `OutboxMessage` carries a unique `EventId` (a `Guid` generated at write time). Consumers use this field as their deduplication key.
2. **Consumer side:** `InboxDeduplicationService` checks `ProcessedInboxMessage` before any handler executes. If the `EventId` has already been recorded, the message is discarded. This makes all handlers idempotent by construction, regardless of their internal logic.

5.2 Error Handling and Dead-Letter Strategy

Table 5.1: Error handling strategy by failure point

Failure point	Behaviour	Operator action
Outbox write fails	Order placement rolls back (same transaction)	None — customer sees failure, can retry
RabbitMQ publish fails	Attempts incremented; retried on next dispatcher tick	Monitor Attempts column; investigate if broker is down
Attempts > N	Row stays in outbox as dead-letter record	Manual intervention: inspect payload, republish or void
Consumer handler throws	Message NACK'd to RabbitMQ; queued or routed to DLQ after configured retries	Inspect RabbitMQ DLQ; fix handler bug; republish
Duplicate message received	Silently discarded by Inbox-Deduplication-Service	None required

5.3 Observability

The system provides three observable surfaces without requiring external tooling:

Outbox table Every cross-context message has a row with **OccurredOnUtc**, **DispatchedOnUtc**, and **Attempts**. The lag between these two timestamps is the propagation delay. Rows with **DispatchedOnUtc** = NULL and high **Attempts** signal delivery problems.

ProcessedInboxMessage table An audit log of every inbound message successfully processed, with timestamps.

Order status transitions The **OrderStatus** field records the current state. Combined with the nopCommerce order notes (added by each handler), this constitutes a state transition audit trail.

RabbitMQ management dashboard Exchange throughput, queue depths, and DLQ contents are visible in the RabbitMQ management UI (<http://localhost:15672> in the Docker Compose environment).

5.4 Security

- RabbitMQ credentials are passed via environment variables (not hardcoded). In the Docker Compose environment they are set in `.env`.
- The outbox table is inside the nopCommerce application database, protected by the same SQL Server access controls as all other tables.
- No new HTTP endpoints are exposed by the two plugins — all communication is DB reads/writes and AMQP.
- The event payload is JSON. No sensitive customer data (payment card numbers, passwords) is included in any cross-context event. Payment operations use nopCommerce’s existing payment service abstractions internally.

5.5 Scalability Constraints

The current design is explicitly single-instance. The in-memory nopCommerce cache (`IStaticCacheManager`) is not shared across instances. Running two nopCommerce processes would cause cache divergence and double-dispatching from the outbox. Horizontal scaling would require Redis (or another distributed cache) and a leader-election mechanism for the dispatcher task. This is acknowledged as out of scope for the demo.

Chapter 6

Traceability Matrix

This chapter traces the path from business drivers through quality attribute scenarios to the concrete implementation artefacts that satisfy them.

6.1 Business Drivers

BD-1 Omnichannel resilience: order acceptance must not depend on warehouse availability.

BD-2 Stock consistency: the online storefront must reflect accurate stock levels within a bounded delay.

BD-3 Customer trust: orders must either succeed reliably or fail cleanly with compensation (no silent data loss, no phantom charges).

BD-4 Operational visibility: operators must be able to observe the state of any order and any in-flight message without custom tooling.

6.2 Quality Attribute Scenarios (Part 1 summary)

Table 6.1: Quality attribute scenarios

ID	Quality	Response measure	Business driver
QA-1	Availability	$\geq 99\%$ order success during 15-min warehouse outage; zero outbox data loss	BD-1
QA-2	Consistency	P95 stock propagation ≤ 30 s; max lag 5 min during RabbitMQ degradation	BD-2
QA-3	Performance	Order placement P95 ≤ 2 s; throughput ≥ 50 orders/min	BD-1, BD-3
QA-4	Recoverability	Outbox drains within 5 min of warehouse recovery	BD-1
QA-5	Operability	Every cross-context message traceable without custom tooling	BD-4

6.3 Full Traceability Matrix

Table 6.2: Business drivers, QA scenarios, and implementation artefacts

BD	QA	Architectural decision	Implementation artefact
BD-1	QA-1	Transactional outbox decouples order acceptance from broker/warehouse	<code>OutboxMessage</code> entity, <code>OutboxEventPublisherDecorator</code> , <code>OrderProcessingServiceDecorator</code>
BD-1	QA-1	<code>TransactionScope</code> ensures outbox row and order commit atomically	<code>OrderProcessingServiceDecorator</code> <code>.PlaceOrderAsync</code>
BD-1	QA-3	<code>ReadCommitted</code> isolation level avoids throughput tax	<code>OrderProcessingServiceDecorator</code> (isolation level option)
BD-1	QA-4	Dispatcher retries outbox rows; recovers automatically after broker restart	<code>OutboxDispatcherTask</code> , <code>RabbitMqPublisher</code> (publisher confirms)
BD-2	QA-2	<code>StockLevelChangedEvent</code> flows from Inventory to Catalog projection	<code>StockLevelChangedHandler</code> , outbox whitelist in <code>OutboxEventPublisherDecorator</code>
BD-3	QA-1	Optimistic reservation: accept first, reconcile asynchronously	<code>OrderPlacedEvent</code> in outbox; <code>StockReservedHandler</code> to <code>Confirmed</code> ; <code>ReservationRejectedHandler</code> to <code>CompensationService</code>
BD-3	QA-1	Compensation on rejection: payment void and customer notification	<code>CompensationService</code> , <code>ReservationRejectedHandler</code> , <code>Compensated</code> order state
BD-4	QA-5	Outbox table as observable message log	<code>OutboxMessage</code> (<code>OccurredOnUtc</code> , <code>DispatchedOnUtc</code> , <code>Attempts</code>)
BD-4	QA-5	Inbox table as audit log of processed messages	<code>ProcessedInboxMessage</code> , <code>InboxDeduplicationService</code>
BD-4	QA-5	Order state transitions traceable via <code>OrderStatus</code> and order notes	<code>StockReservedHandler</code> , <code>ReservationRejectedHandler</code> , <code>ShipmentDispatchedHandler</code>
BD-1, BD-4	QA-5	Outbox retention prevents unbounded growth	<code>OutboxRetentionTask</code> (7-day cleanup)

6.4 Risk Coverage

Table 6.3: Risk register: implementation status

#	Risk	Status	Mitigation implemented
R1	Whitelist collides with unrelated consumers	Mitigated	Explicit whitelist in <code>OutboxEventPublisherDecorator</code> ; all other events pass through
R2	No ambient <code>TransactionScope</code> in <code>PlaceOrderAsync</code>	Mitigated	<code>OrderProcessingServiceDecorator</code> supplies the scope
R3	Dispatcher cadence vs. QA-2 propagation target	Open	Cadence set to 5-10 s; measure under load in demo
R4	No horizontal scaling (in-memory cache)	Acknowledged	Single-instance demo; Redis is future work
R5	Outbox table grows unbounded	Mitigated	<code>OutboxRetentionTask</code> deletes dispatched rows older than 7 days

Chapter 7

Demo Orchestration

7.1 Prerequisites

All services are started via Docker Compose from the repository root:

Listing 7.1: Start the full environment

```
1 docker compose up -d
```

Wait for all services to report healthy, then open:

- nopCommerce storefront: <http://localhost:5000>
- nopCommerce admin: <http://localhost:5000/admin>
- RabbitMQ management: <http://localhost:15672> (guest/guest)

Open a SQL client connected to the nopCommerce SQL Server database to observe the `OutboxMessage` and `ProcessedInboxMessage` tables live.

7.2 Scenario 1 — Happy Path

Proves: end-to-end flow works; outbox is the reliability seam.

1. Browse the storefront, add a product to the cart, proceed to checkout.
2. Complete the order. Observe the order confirmation page.
3. In the SQL client, run:

```
1 SELECT * FROM OutboxMessage ORDER BY OccurredOnUtc DESC;
```

Verify a row with `MessageType = 'OrderPlacedEvent'` and `DispatchedOnUtc` IS NULL appears immediately after placement, then `DispatchedOnUtc` is set within 5-10 s.

4. In the RabbitMQ dashboard, confirm a message was consumed from the `order.placed` queue.
5. In the admin panel (**Orders**), confirm the order transitions from **Placed** to **Confirmed** within seconds.
6. In `ProcessedInboxMessage`, confirm a row for `StockReservedEvent` is recorded.

Expected result: order in **Confirmed** state; outbox row dispatched; no errors in logs.

7.3 Scenario 2 — Warehouse Degradation

Proves: order acceptance is decoupled from warehouse availability (QA-1 mandatory pressure point).

1. Stop the Inventory Sync Service:

```
1 docker compose stop inventory-sync
```

2. Place two or three orders via the storefront. Each should receive an order confirmation immediately.
3. In the SQL client, observe `OutboxMessage`: rows accumulate with `DispatchedOnUtc` IS NULL (for the `StockReservedEvent` / `OrderPlacedEvent` rows pending consumption) or `DispatchedOnUtc` is set (the outbox dispatched to RabbitMQ, but the Inventory service is not consuming).
4. In the RabbitMQ dashboard, observe the `order.placed` queue depth growing.
5. In the admin panel, orders remain in **Placed** state — not confirmed, not cancelled.

Expected result: storefront remains 100% available to customers; outbox accumulates messages; orders are queued, not lost.

7.4 Scenario 3 — Recovery

Proves: the system self-heals when the warehouse recovers (QA-4).

1. Restart the Inventory Sync Service:

```
1 docker compose start inventory-sync
```

2. Watch the RabbitMQ `order.placed` queue drain in the management dashboard.
3. In the SQL client, observe `ProcessedInboxMessage` rows being added as the Inventory service processes the backlog.

4. In the admin panel, observe the orders transitioning from `Placed` to `Confirmed` within minutes.

Expected result: all queued orders reconcile within 5 minutes of restart; no data loss; no manual intervention required.

If Scenarios 2 and 3 work live in the presentation, the mandatory runtime challenge requirement is satisfied.

7.5 Scenario 4 — Forced Rejection and Compensation

Proves: the compensation flow works; the `Compensated` state is reachable.

1. Configure the Inventory Sync Service (or its OpenBoxes stub) to return a rejection response for the next reservation request. (See the WireMock / stub configuration in `docker/wiremock/mappings/.`)
2. Place an order via the storefront.
3. Observe the `ReservationRejectedEvent` arrive in the `ProcessedInboxMessage` table.
4. In the admin panel, confirm the order transitions to `Compensated`.
5. Confirm the payment was voided (check the order payment status).
6. Confirm a customer notification was sent (check the `noCommerce` message queue log in admin).

Expected result: order in `Compensated` state; payment voided; customer notified; no orphaned charge.

7.6 Scenario 5 — Audit Trail

Proves: the system is observable without custom tooling (QA-5).

1. In the SQL client:

```
1  -- All outbox messages with dispatch lag
2  SELECT MessageType ,
3             OccurredOnUtc ,
4             DispatchedOnUtc ,
5             DATEDIFF(SECOND, OccurredOnUtc , DispatchedOnUtc) AS
6             LagSeconds ,
7             Attempts
8  FROM OutboxMessage
9  ORDER BY OccurredOnUtc DESC;
```

```
10 -- All processed inbound messages
11 SELECT * FROM ProcessedInboxMessage ORDER BY ProcessedOnUtc
    DESC;
```

2. In the RabbitMQ dashboard, show the Dead Letter Queue (DLQ) — should be empty in the happy path.
3. In the admin panel, open an order and show the order notes tab — each state transition is recorded with a timestamp.

Expected result: every message and every state transition is traceable from standard operator surfaces.

Chapter 8

Tradeoffs, Scope Cuts, and Limitations

8.1 Tradeoffs We Own

8.1.1 At-Least-Once vs. Exactly-Once Delivery

We chose at-least-once delivery. Exactly-once would require a distributed transaction across the SQL Server database and RabbitMQ, which RabbitMQ does not support as an XA resource. The consequence is that every consumer must be idempotent, we enforce this centrally via the inbox deduplication table (ADR-6). Handlers are shielded from this concern at the cost of an extra DB read per inbound message.

The alternative, trying to achieve exactly-once via RabbitMQ quorum queues and deduplication windows, reduces but does not eliminate duplicates, and introduces a time-windowed assumption that is harder to reason about than a persistent record. We prefer the persistent record.

8.1.2 Dispatcher Polling vs. DB Change Notifications

The outbox dispatcher is a polling `IScheduleTask` (5-10 s cadence). This introduces a minimum propagation lag equal to the polling interval. A push-based mechanism (SQL Server `QUERY_NOTIFICATION` or Debezium CDC) would reduce lag to near-zero but adds significant operational complexity and ties the design to SQL Server (ADR-1, alternative C).

For QA-2's 30 s P95 target, a 5-10 s poll is comfortable. For QA-3's 2 s order placement P95, the polling happens in the background and is not on the critical path. The tradeoff is acceptable for the demo scale.

8.1.3 Whitelisted vs. Universal Outbox

The `OutboxEventPublisherDecorator` whitelists only the six cross-context event types. All other events (including Brevo/Omnisend/nopCommerce-internal events) pass through unchanged. This avoids breaking existing plugins but means adding a new cross-context event requires a code change to the whitelist. We consider this deliberate friction: cross-

context events represent API surface between bounded contexts and should not be added carelessly.

8.1.4 In-Process Outbox vs. Separate Worker

The outbox dispatcher and the consumer subscriber run inside the nopCommerce process as `IScheduleTask` instances. A separate worker process would allow independent scaling and deployment. We rejected this because it adds a new deployable unit for a demo-scale system where the single-instance constraint is already acknowledged (Risk R4). The `IScheduleTask` approach keeps the deployment simple at the cost of co-location with the web process.

8.2 Scope Cuts and Their Consequences

Table 8.1: Scope cuts and their consequences

Scope cut	Consequence / what breaks if we change our minds
No Redis / no distributed cache	Adding a second nopCommerce instance would cause cache divergence and double-dispatching. Requires replacing <code>IStaticCacheManager</code> with a Redis-backed implementation.
Payment = Order Management	No separate Payment service means payment operations go through nopCommerce’s payment abstractions directly. Extracting payment later would require its own event contracts and compensation flows.
No federated SSO	Each channel maintains its own customer identity. Cross-channel customer recognition is not supported.
Outbox in the monolith process	If the nopCommerce process crashes mid-dispatch, in-flight messages are retried on restart (at-least-once). No data is lost, but the recovery window equals the process restart time.
Carrier = WireMock simulation	Real carrier integration would require a non-stub ACL layer in the Shipping service. The event contracts (<code>ShipmentDispatchedEvent</code>) are already correctly shaped for this.

8.3 Known Limitations

1. **Load testing at demo scale only:** A k6 script (`load-tests/order-placement.js`) validates QA-3 ($P95 \leq 2$ s, throughput ≥ 50 orders/min) against the Docker Compose environment. The same script drives the live stress demo (Scenarios 2 and 3: kill the inventory service mid-run, restart it, observe recovery). Results are representative of a single-instance deployment; they do not model horizontal scaling, a Redis-backed cache, or a clustered RabbitMQ broker. QA-2 propagation lag ($P95 \leq 30$ s) is measured indirectly by querying the `OutboxMessage` dispatch lag after the run rather than inline in k6. This is Risk R3.

2. **No circuit breaker:** the outbox dispatcher makes no attempt to detect a persistently down broker and back off intelligently. It retries on every tick. A circuit breaker (or exponential backoff on `Attempts`) would reduce noise in production logs.
3. **No schema versioning on event payloads:** event payloads are JSON with no version field. Schema evolution (adding/removing fields) is handled by convention (additive changes only). A formal schema registry was out of scope.
4. **Manual DLQ handling:** messages that exceed the maximum dispatch attempts remain in the outbox table as a dead-letter view. There is no automated alerting or reprocessing UI. An operator must query the table and manually republish or void.
5. **Single RabbitMQ instance:** no clustering, no mirrored queues. RabbitMQ is a single point of failure in the demo environment. In production, quorum queues and a clustered broker would be required.

8.4 What We Would Do Differently at Production Scale

1. Replace the polling dispatcher with a push-based mechanism (CDC or SQL Server change notifications) to reduce propagation lag and DB read load.
2. Move the dispatcher and consumer to a separate worker process, enabling independent scaling and deployment.
3. Add Redis as a distributed cache to allow multiple nopCommerce instances.
4. Introduce exponential backoff and a proper DLQ with monitoring alerts for failed outbox messages.
5. Add a schema registry (or at minimum, a version field on all event payloads) to enable safe schema evolution.
6. Implement RabbitMQ quorum queues and broker clustering for message durability.

Chapter 9

Use of Large Language Models

9.1 Overview

The team used Claude by Anthropic as a productivity tool. This chapter documents how LLMs were used, where our judgement remained essential, and what we learned about effective human-LLM collaboration in a software architecture project.

The guiding principle was **augmentation, not replacement**: every architectural decision, every design choice, and every line of production code was understood, reviewed, and owned by the team. LLMs were used to accelerate execution after decisions had been made, not to make decisions on our behalf.

9.2 Where LLMs Were Used

9.2.1 Documentation and Report Writing

The most significant use was in drafting the report. We provided the architectural decisions, implementation outcomes, and reasoning; Claude was used to translate these into structured LaTeX prose, maintain consistency across chapters, and ensure the report accurately reflected the codebase.

This required active collaboration: draft sections were reviewed against the actual implementation, corrected where they diverged, and refined through multiple iterations. The LLM was particularly useful for cross-cutting chapters such as the traceability matrix (Chapter 6) and the cross-cutting concerns chapter (Chapter 5), where synthesising information from multiple parts of the codebase into a coherent narrative was time-consuming.

9.2.2 Architecture Diagram Generation

Mermaid diagrams were generated iteratively with LLM assistance. We described the system behaviour, for example the full order lifecycle including the happy path, degradation, recovery, and compensation, and the LLM produced Mermaid source code that was then reviewed, corrected, and refined. This was faster than writing Mermaid syntax by

hand, but every diagram was validated against the actual implementation before being included in the report or presentation.

9.2.3 Code Generation

The majority of production code was written by Claude based on our detailed specifications. We described the required behaviour, the nopCommerce extension points to use, and the constraints to respect (e.g., idempotency requirements for the inbox), and Claude produced implementation drafts. These were then reviewed against the architectural intent, tested against the QA scenarios, and corrected where they diverged from expected behaviour.

9.2.4 Architectural Reasoning

Claude was used as a sounding board during architectural discussions, for example, validating the rationale for splitting one plugin into two, or explaining why the **Compensated** order state could not reuse an existing **OrderStatus** value without losing semantic precision. In these cases the LLM did not drive the decision; it helped articulate and stress-test reasoning that the team had already developed.

9.3 Where LLMs Were Not Used

The following were entirely human-authored:

- All architectural decisions: the transactional outbox pattern, the plugin split, the inbox deduplication strategy, the **Compensated** state design
- The feasibility spike and its conclusions
- All code review, functional testing, and validation against QA scenarios, every generated artefact was verified by the team before being accepted

9.4 Reflection

Overall, LLM assistance significantly accelerated both implementation and documentation. Code generation was the highest-leverage use: rather than writing boilerplate and plumbing by hand, the team could focus on specifying the correct behaviour and validating that the generated code delivered it. The architecture and the decisions behind it remain the team's own; the LLM was an executor, not a designer.